CS 331 Midterm 2 Cheat Sheet

Nov 11, 2026 · @Aryianah Wilson

Graphs

Algorithm	Data structure	Works when	Runtime	Use it for
BFS	Queue (oldest out first)	Any graph	O(m + n)	Unweighted shortest paths, connected components
DFS	Stack (newest out first)	Any graph	O(m + n)	Topological order (fails = not a DAG), strongly connected components
Dijkstra (greedy)	Heap	Weights ⩾ 0	O((m + n) log n)	Shortest paths, non-negative weights
Bellman-Ford (DP)	Relax all edges n − 1 rounds	Negative weights OK	O(mn)	Shortest paths; nth round still improves → negative cycle

- Negative weights are fine; negative cycles break shortest paths.
- Relax edge (u, v): if d(u) + w(u, v) < d(v), set d(v) = d(u) + w(u, v).
- Dijkstra works because it extracts the smallest estimate and weights are positive, so no later path can beat it. Extracted values never decrease (ties possible).
- Shortest-path predecessor: if x is right before u on a shortest path, d(x) = d(u) − w(x, u) < d(u), so x is extracted before u.

Modifying Dijkstra (Practice P1 template)

1. Track the extra quantity in its own array, or use lexicographic pairs (d, ℓ) as heap keys.
2. `<` case: reset the extra quantity from the new path. `=` case (tie): keep the better one; heap unchanged.
3. Runtime: O(1) extra per edge, so still O((m + n) log n).
4. Proof: cite Dijkstra's correctness, then induction on extraction order; prove ⩾ and ⩽ separately.
5. Real-valued weights: don't encode as w·N + 1; use pairs.

Graph search algorithms

```
BFS(G, s):                      // queue: oldest out first
  dist[v] = ∞ for all v; dist[s] = 0
  Q = queue with s
  while Q not empty:
    u = Q.dequeue()
    for each edge (u, v):
      if dist[v] = ∞:
        dist[v] = dist[u] + 1
        Q.enqueue(v)
  return dist                   // fewest edges from s; O(m + n)

DFS(G):                         // stack / recursion: newest first
  visited[v] = false for all v; L = empty list
  for each v: if not visited[v]: Explore(v)
  Explore(u):
    visited[u] = true
    for each edge (u, v): if not visited[v]: Explore(v)
    L.prepend(u)                // u finishes → front of L
  return L                      // O(m + n)
```

- **Topological order:** L from DFS (reverse finish order). Check every edge (u, v) has u before v in L; any edge pointing backward → cycle → not a DAG.
- **Connected components (undirected):** run BFS/DFS from each unvisited vertex; each run = one component. O(m + n).
- **Strongly connected components (directed):** DFS on G to get finish order L; then DFS on reversed G, starting vertices in L′s order; each tree = one SCC. O(m + n).
- **Modifying search:** change what goes in the bag, what you store per vertex (dist, parent, count), or which edges you′re allowed to follow; runtime stays O(m + n) if each vertex/edge is processed O(1) times.

Flow

Hint: ″any polynomial runtime″ usually means reduce to max flow.

- Net flow: ðf(v) = (flow out of v) − (flow into v).
- s-t max flow: maximize ðf(s), subject to 0 ⩽ f(e) ⩽ c(e) on every edge and ðf(v) = 0 for every v ≠ s, t.

- Ford-Fulkerson: while the residual graph has an s → t path, push flow along it. No path = max flow.
- Integer capacities → an integer max flow exists.
- Flow decomposition: any flow splits into s → t paths (plus cycles you can drop). Integer flow of value k → k paths carrying 1 unit each.

Which max flow algorithm

- Default: Edmonds-Karp (Ford-Fulkerson + BFS for augmenting paths). $O(V \cdot E^2)$, always polynomial; cite as lecture.
- Plain Ford-Fulkerson: $O(E \cdot F)$, F = max flow value. NOT polynomial if F can be huge (e.g. F = total soldiers). Only use when F is small (e.g. $F \leqslant n$).
- Runtime: count your network's V and E, plug into $O(V \cdot E^2)$.
- Capacity ∞: fine, or replace with a big-enough number (e.g. total supply).

Bipartite matching: s → each left vertex (capacity 1), left → right on original edges (capacity 1), each right vertex → t (capacity 1). Max flow value = max matching size.

Capacity tricks

- "Vertex v takes up to k" → the s → v (or v → t) edge gets capacity k.
- "Each pair used at most once" → middle edges capacity 1.
- Limit on a middle vertex → split v into v_in → v_out with that capacity.

Full-credit proof (both directions)

1. Every valid solution to the original problem → a flow of the same value.
2. Every integer flow → apply flow decomposition → each unit path gives one piece of a valid solution of the same value.

Generic flow answer template

1. **Network:** list vertices (s, t, copies of items, e.g. left/right or before/after) and each edge with its capacity (source edges = supply / "up to k"; sink edges = demand; middle edges = allowed choices, capacity 1 or ∞).
2. **Algorithm:** build the network, run Edmonds-Karp (lecture), return True / the answer iff max flow = target value (usually sum of sink capacities).
3. **Runtime:** build $O(n + m)$ + Edmonds-Karp $O(V \cdot E^2)$ with V, E of the NEW network + check $O(n)$. Total = Edmonds-Karp term.
4. **Proof (⇒) solution → flow:** define f on every edge from the solution. Check $0 \leqslant f \leqslant c$ on every edge, $\eth f(v) = 0$ for all $v \neq s, t$ (flow in = flow out), and value = target.

5. **Proof (⇐) flow → solution:** capacities are integers → an integer max flow exists. Read the solution off the flow (flow on a middle edge = how much of that choice), or apply flow decomposition into unit s → t paths. Capacities enforce every constraint; sink edges full → every demand met exactly.
6. **Conclude:** a valid solution exists iff max flow = target.

Example (Practice P2, soldiers): s → i_L cap A[i]; i_L → i_R cap ∞ (stay); i_L → j_R cap ∞ for road (i, j); j_R → t cap B[j]. True iff ΣA = ΣB and max flow = ΣB. V = 2n + 2, E = m + 3n → $O(n(m + n)^2)$.

Continuous

Property	Definition	1-D derivative test	Piecewise check
Convex	Graph stays at or below every chord; $f(y) \geq f(x) + f'(x)(y - x)$ (above every tangent)	$f'' \geq 0$	Each piece convex AND slope never drops at a joint
L-Lipschitz (f doesn't change too fast)	$\lvert f(x) - f(y)\rvert \leq L \cdot \lvert x - y\rvert$	$\lvert f'\rvert \leq L$ everywhere	Largest |slope| over all pieces ≤ L
β-smooth (slope doesn't change too fast)	$\lvert f'(x) - f'(y)\rvert \leq \beta \cdot \lvert x - y\rvert$	$f'' \leq \beta$ (in absolute value)	f' continuous (no kinks/corners) AND $\lvert f''\rvert \leq \beta$ on each piece

- To show a label fails: give a counterexample (two points x, y that break the inequality, or a kink for smoothness).
- A corner like |x| at 0 breaks smoothness, not convexity.

Power rule: $d/dx\ [c \cdot x^n] = c \cdot n \cdot x^{n-1}$. Constants → 0. Example: $3x^4 \rightarrow 12x^3$.

Chain rule: $d/dx\ [g(h(x))] = g'(h(x)) \cdot h'(x)$. Shortcut: $d/dx\ (x + a)^n = n(x + a)^{n-1}$. Example: $(x + 5)^3 \rightarrow 3(x + 5)^2$. Also: |x| has slope −1 for x < 0, +1 for x > 0.

Quotient rule: $d/dx\ [g(x)/h(x)] = (g'(x) \cdot h(x) - g(x) \cdot h'(x)) / h(x)^2$. Example: $x/(x + 1) \rightarrow 1/(x + 1)^2$.

How to check convexity (piecewise)

1. Find f' on each piece (power rule).
2. Find f'' on each piece; every piece needs $f'' \geqslant 0$.
3. Joints = the x values where the formula switches (from the conditions, e.g. $x \leqslant a$, $a \leqslant x \leqslant b$).
4. At each joint: left slope = joint plugged into f' of the piece just below it; right slope = joint plugged into f' of the piece just above it.
5. Need left slope $\leqslant$ right slope at every joint (slope never drops).
6. Steps 2 and 5 both pass → convex. A slope drop at a joint → not convex (downward corner, like $-|x|$ at 0).

Joint slope rules

Property	At every joint	Why
Convex	left slope $\leqslant$ right slope	Slope may jump up, never down
Smooth	left slope = right slope	Any jump (up or down) breaks $\lvert f'(x) - f'(y)\rvert \leqslant \beta\lvert x - y\rvert$

Example: $|x|$ is convex (slope $-1 \to 1$, goes up) but not smooth (slope jumps at 0).

Not-convex counterexample (chord test)

1. Pick two points a, b inside a piece where $f'' < 0$.
2. Midpoint = $(a + b)/2$, halfway between a and b ($a = 1$, $b = 3 \to 2$); keep a, b, and the midpoint in the same $f'' < 0$ piece. Chord height at the midpoint = $(f(a) + f(b))/2$.
3. Show $f((a + b)/2) > (f(a) + f(b))/2$ → graph goes above the chord → not convex.

SVD: $M = U\Sigma V^T$. Σ is diagonal with entries $\geqslant 0$ (singular values). U and V have orthonormal columns: $U^TU = I$, $V^TV = I$.

PSD: symmetric M with $x^TMx \geqslant 0$ for all x; equivalently, all eigenvalues $\geqslant 0$.

Symmetric: $M_{ij} = M_{ji}$ for all i, j (mirror across the main diagonal; compare each entry above the diagonal with its mirror below).

PSD quick tests

1. Not symmetric → not PSD (name one mismatched pair).
2. Negative diagonal entry $M_{ii} < 0$ → not PSD ($x = e_i$ gives $x^TMx = M_{ii} < 0$).

3. A 2×2 block with $M_{ii} \cdot M_{jj} < M_{ij}^2$ → not PSD.
    - Test every diagonal pair by multiplying them and seeing if it's smaller than the outer corners being squared (outer corner = M_{ij}, row i, column j; 6 pairs in a 4×4). Only rules OUT: all pairs passing does not prove PSD.
    - Formula (x nonzero only in positions i, j): $x^T M x = M_{ii} \cdot x_i^2 + 2 \cdot M_{ij} \cdot x_i \cdot x_j + M_{jj} \cdot x_j^2$
    - Witness x: same length as the matrix (4 entries for 4×4). Put 0 in every position except i and j. For block [[a, b], [b, c]] (a = M_{ii}, b = M_{ij}), set $x_i = b$, $x_j = -a$. Example: pair (3, 4) with a = 2, b = 1 → x = (0, 0, 1, −2); pair (1, 2) → x = (b, −a, 0, 0). This gives $x^T M x = a(ac - b^2) < 0$.
4. To prove PSD: write $M = vv^T$ (or a sum of these); then $x^T M x = (v^T x)^2 \geq 0$.

Proving PSD with vv^T

- v = a column vector (e.g. (2, 1) stacked vertically); v^T = the same numbers as a row. $v^T x$ = dot product = one number.
- vv^T = column × row = a matrix (a "multiplication table" of v): entry (i, j) = $v_i \cdot v_j$. Diagonal = v_i^2.
- Find v: diagonal entries are perfect squares and every row is a multiple of the same row.
- Example: M = all 9s (4×4). $M = vv^T$ with v = (3, 3, 3, 3). For any x, $x^T M x = (v^T x)^2 \geq 0$, so M is PSD.
- Other cases: sum $M = v_1 v_1^T + v_2 v_2^T \rightarrow x^T M x = (v_1^T x)^2 + (v_2^T x)^2 \geq 0$. Diagonal with entries $\geq 0 \rightarrow x^T M x = \Sigma M_{ii} x_i^2 \geq 0$.

Randomized

- Linearity of expectation: E[X + Y] = E[X] + E[Y], even when X and Y are dependent.
- Union bound: Pr[A or B] ⩽ Pr[A] + Pr[B], even when A and B are dependent.
- Basic boosting: an algorithm that fails with probability p, repeated k independent times, fails every time with probability p^k.

3-stage plan (estimators)

1. Basic estimator: random X with E[X] = the true answer.
2. Mean boosting: average k independent copies; variance drops by a factor of k. Chebyshev → within ε with probability ⩾ 2/3.
3. Median boosting: take the median of O(log(1/δ)) runs of stage 2. Chernoff → fails with probability ⩽ δ.

Check with notes: the cut-off line "basic interval by…" may be "by Chebyshev."

Practice Midterm Solutions

P1: Shortest shortest paths (Dijkstra variant)

Question: Modify Dijkstra (positive weights, s reaches all of V) to return (d(v), ℓ(v)) for all v, where d(v) = least total weight of an s-v path and ℓ(v) = fewest edges among s-v paths of weight d(v). Analyze the runtime.

Algorithm. Run Algorithm 1 plus an array ED (size n), ED[v] = ∞, ED[s] = 0. Relaxing (u, t):

- If u.val + w(u,t) < t.val: ED[t] ← ED[u] + 1, then Delete/Insert as usual.
- Else if u.val + w(u,t) = t.val and ED[u] + 1 < ED[t]: ED[t] ← ED[u] + 1 (heap unchanged).
- Return (D[v], ED[v]) for all v.

Runtime. O(n) setup/output + O(1) extra per edge = O(m + n) extra. Heap ops unchanged → O((m + n) log n).

Proof. D[v] = d(v) by Dijkstra's correctness (lecture). Show ED[u] = ℓ(u) at extraction, by induction on extraction order. Base: ED[s] = 0 = ℓ(s).

- **ED[u] ⩾ ℓ(u):** ED[t] is always the edge count of a real s → t path of weight t.val. At extraction u.val = d(u), so ED[u] counts a shortest path → ED[u] ⩾ ℓ(u).
- **ED[u] ⩽ ℓ(u):** let P = shortest s → u path with ℓ(u) edges, x = vertex before u. d(x) = d(u) − w(x,u) < d(u) (w > 0), so x is extracted before u. Prefix of P → ℓ(x) ⩽ ℓ(u) − 1; by IH ED[x] = ℓ(x). Relaxing (x, u) gives candidate weight d(u) → ED[u] ⩽ ℓ(x) + 1 ⩽ ℓ(u). Afterward `<` can't fire (u.val = d(u)) and `=` only lowers ED[u]. ∎

P2: Soldiers (flow)

Question: Directed graph G = ([n], E), |E| = m (cities + one-way railroads). City i has A[i] soldiers; in one relocation, an integer number of soldiers can move from i to j along a road (i, j). Need exactly B[i] soldiers at every city afterward. Give a polynomial-time algorithm returning True/False: does a successful relocation plan exist?

Algorithm. If ΣA ≠ ΣB, return False (soldiers are conserved). Build: s → i_L cap A[i]; i_L → i_R cap ∞ (stay); i_L → j_R cap ∞ for each road (i, j); j_R → t cap B[j]. Run Edmonds-Karp. Return True iff max flow = ΣB.

Runtime. V = 2n + 2, E = m + 3n. Build O(n + m), Edmonds-Karp O(V·E²) → O(n(m + n)²).

Proof.

- **Plan → flow:** x_{ij} soldiers move i → j, $stay_i = A[i] - \Sigma_j x_{ij} \geq 0$ stay. Set f(s → i_L) = A[i], f(i_L → j_R) = x_{ij}, f(i_L → i_R) = $stay_i$, f(j_R → t) = B[j]. Capacities hold. At i_L: in A[i] = out Σx_{ij} + $stay_i$. At j_R: in = soldiers ending at j = B[j] = out. Value ΣB.
- **Flow → plan:** integer capacities → integer max flow of value ΣB = ΣA, so every s-edge and t-edge is full. Move f(i_L → j_R) soldiers from i to j; f(i_L → i_R) stay. Conservation at i_L: exactly A[i] soldiers leave/stay at city i. At j_R: exactly B[j] end at city j. Integer amounts → valid plan.
- Plan exists iff max flow = ΣB. ∎

P3: Convex / 1-Lipschitz / 1-smooth

Question: For each f, which labels apply: (a) convex, (b) 1-Lipschitz, (c) 1-smooth? Briefly justify each label that applies, or give a counterexample showing it doesn't.

(i) f = 0 ($x \leq 0$), $\frac{1}{2}x^2$ ($0 \leq x \leq 2$), $2x - 2$ ($x \geq 2$). $f' = 0, x, 2$. $f'' = 0, 1, 0$.

- **Convex — yes.** $f'' \geq 0$ on every piece; f' = 0 both sides of x = 0 and 2 both sides of x = 2, so f' non-decreasing.
- **1-Lipschitz — no.** $f' = 2 > 1$ for $x \geq 2$. $f(3) - f(2) = 4 - 2 = 2 > 1 = |3 - 2|$.
- **1-smooth — yes.** $|f''| \leq 1$ on every piece and f' has no jumps, so $|f'(x) - f'(y)| \leq |x - y|$.

(ii) $f = 3/2 - \frac{1}{2}(x + 2)^2$ ($x \leq -1$), $|x|$ ($-1 \leq x \leq 1$), $3/2 - \frac{1}{2}(x - 2)^2$ ($x \geq 1$). $f' = -x - 2, x/|x|$ (± 1), $-x + 2$. $f'' = -1, 0, -1$.

- **Convex — no.** $f'' = -1 < 0$ for $x \geq 1$. f(1) = f(3) = 1 but f(2) = 3/2 > 1 = chord midpoint.
- **1-Lipschitz — no.** $f' = -x - 2$ unbounded for $x \leq -1$. f(−3) = 1, f(−4) = −½ → difference 3/2 > 1.
- **1-smooth — no.** f' jumps −1 → +1 at x = 0. x = −0.1, y = 0.1: $|f'(x) - f'(y)| = 2 > 0.2$.

P4: PSD matrices

Question: Which of these 4×4 matrices are PSD? Justify. (i) rows (15 3 1 3), (3 −2 7 5), (1 7 8 −1), (3 5 −1 10). (ii) all 9s. (iii) rows (6 5 4 3), (5 4 3 2), (4 3 2 1), (3 2 1 0). (iv) rows (2 2 0 0), (1 2 0 0), (0 0 1 0), (0 0 1 1).

- **(i) Not PSD:** $M_{22} = -2 < 0$; $x = e_2$ gives $x^TMx = -2$.
- **(ii) PSD:** $M = vv^T$, v = (3, 3, 3, 3). For any x, $x^TMx = (v^Tx)^2 \geq 0$.
- **(iii) Not PSD:** pair (3, 4): $M_{33} \cdot M_{44} = 0 < 1 = M_{34}^2$. x = (0, 0, 1, −2) gives $x^TMx = -2$.
- **(iv) Not PSD:** not symmetric, $M_{12} = 2 \neq 1 = M_{21}$.

P5: Streetlights (randomized)

Question: n streetlights (n even), exactly n/2 malfunctioning. Normal lights are on every night. Each night, one malfunctioning light (uniformly at random, re-randomized nightly) is off; the rest are on. You inspect exactly one light per night. Give T and a strategy so that after T days you've found all malfunctioning lights with probability ≥ 3/4. Any finite T with a correct proof gets full credit.

Strategy. Let $k = \lceil (n/2)\cdot\ln(2n) \rceil$. Inspect light 1 for k nights in a row, then light 2 for k nights, …, light n. Declare a light malfunctioning iff it was ever seen off. **$T = n\cdot k = n\lceil (n/2)\ln(2n) \rceil$.**

Proof.

- Normal lights are always on → never wrongly flagged.
- A malfunctioning light is the off one on a given night w.p. 1/(n/2) = 2/n, independently each night (re-randomized). Missed in all k of its nights w.p. $(1 - 2/n)^k \leq e^{(-2k/n)} \leq e^{(-\ln(2n))} = 1/(2n)$, using $1 - y \leq e^{(-y)}$.
- Union bound over the n/2 malfunctioning lights (works even though the events are dependent): Pr[some light missed] ≤ (n/2)·(1/(2n)) = 1/4.
- So all malfunctioning lights are found w.p. ≥ 3/4. ∎

Pattern: repeat a test k times to make one item's failure tiny (basic boosting: p^k), then union bound over all items.